BUỔI 09

Continuous Delivery (CD) & triển khai tự động

Tự động hoá pipeline từ GitHub Actions lên AWS EC2 / VPS & quản lý bảo mật



Người trình bày

Đào Trọng Hiếu · Trung tâm Đào tạo AI Chuyên sâu

Mục tiêu buổi học

01

Tư duy & kiến trúc

- Hiểu bản chất của **CD** - tự động hoá bước đưa ứng dụng ra máy chủ
- Phân biệt **Continuous Delivery** với **Continuous Deployment**
- Tư duy bảo mật **thông tin nhạy cảm** trong pipeline

02

Kỹ năng thực hành

- Cấu hình **GitHub Secrets**
- Viết pipeline **push Docker image** lên Registry
- Triển khai **SSH tự động** lên EC2 / VPS
- Debug **log** khi triển khai lỗi

Vị trí Buổi 09 trong lộ trình đào tạo

Buổi 07 · đã xong

CI - Tích hợp liên tục

Push code, unit test, build Docker image.

Buổi 08 · đã xong

AWS EC2 & Cloud Infrastructure

Khởi tạo máy chủ, Security Group, SSH Key.

Buổi 09 · hôm nay

CD - Continuous Delivery

Nối chuỗi tự động, đẩy ứng dụng chạy **live trên AWS EC2** đã tạo ở Buổi 8.

Bài toán thực tế: Deploy thủ công vs Deploy tự động (CD)

Thao tác tay (Manual)

- SSH thủ công, gõ lệnh kéo code - dễ gõ sai
- Mất **30-40 phút / lần**
- Phụ thuộc cá nhân, khó bàn giao

Tự động (CD)

- Push code là tự chạy, chuẩn hoá 100%
- Mất **< 3 phút**
- Kiểm soát và rollback dễ dàng

Phân biệt Continuous Delivery vs Continuous Deployment

Continuous Delivery

Tự động build, test và đóng gói. Bước đẩy lên Production cần **1 nút bấm duyệt thủ công** (Manual Approval).

Continuous Deployment

Mọi commit qua kiểm thử thành công đều **lập tức tự động** đẩy ra Production, không cần con người can thiệp.

Lựa chọn hôm nay: xây pipeline **Continuous Delivery** lên môi trường Staging / Production.

Toàn cảnh kiến trúc CD Buổi 09 (Full Pipeline)



Rủi ro lọt lộ thông tin nhạy cảm

Ghi cứng (Hardcode) vào file .yml / source code

Push lên Git là lộ ngay - lịch sử commit giữ vĩnh viễn.

```
API_KEY: "sk_live_123456"
```

In thử biến bí mật ra log

Ai xem được log là thấy ngay giá trị thật.

```
run: echo $DB_PASSWORD
```

Hậu quả: bị **bot quét tự động lấy key**, mất tiền Cloud, rò rỉ dữ liệu.

KHỐI 2 · Bảo mật với GitHub Secrets

GitHub Secrets là gì & cơ chế hoạt động

Kho lưu trữ khoá bí mật được mã hoá, cấp độ Repository / Organization.

1 · Nhập & mã hoá

Nhập Secret vào Settings - GitHub mã hoá bất khả đảo ngược.

2 · Nạp vào RAM Runner

Workflow chạy mới giải mã, nạp làm biến môi trường - không lưu ổ đĩa.

3 · Tự động che (Masking)

Vô tình in ra log, GitHub tự đổi giá trị thành ***.

Phân loại Secrets trên GitHub

Organization

Dùng chung cho **toàn bộ dự án trong công ty**.

Repository

Áp dụng cho **toàn bộ branch / workflow** trong repo đó - loại ta dùng hôm nay.

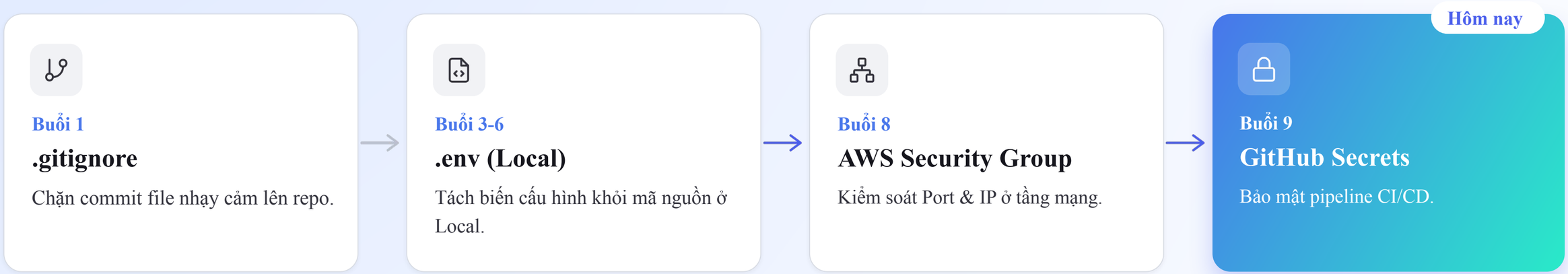
Environment

Riêng cho từng môi trường (**staging, production**), có thể đòi hỏi quyền duyệt (Reviewers).

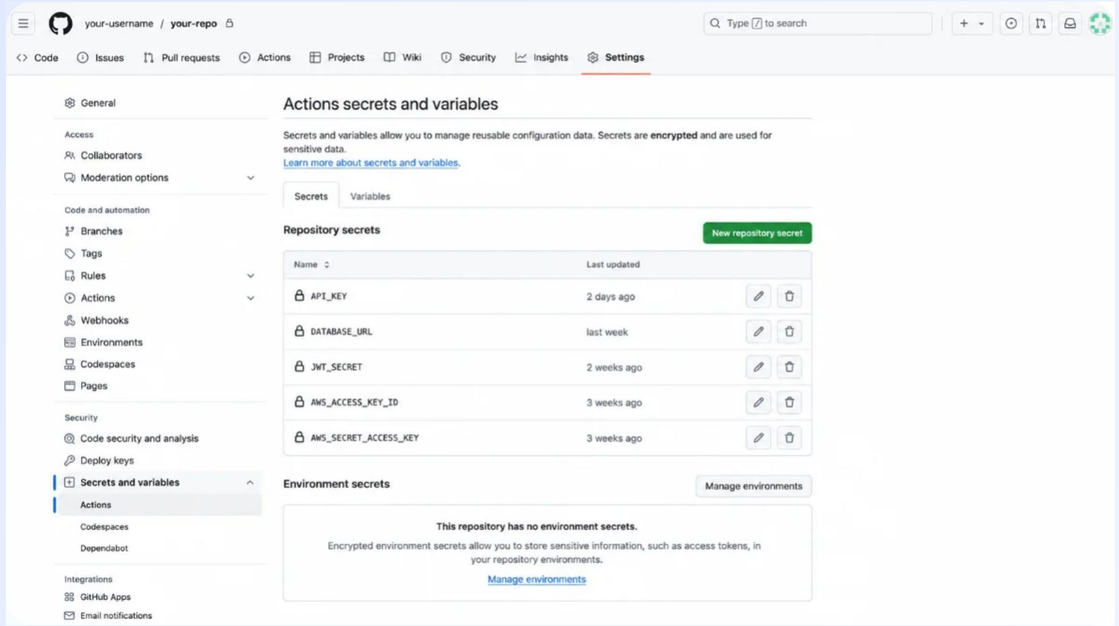
KHỐI 2 · Bảo mật với GitHub Secrets

Sợi chỉ bảo mật xuyên suốt khoá học DevOps

Một tư duy bảo vệ bí mật, lặp lại ở mọi tầng của dự án.



Thực hành: khai báo Secrets trên giao diện GitHub



Settings › Secrets and variables › Actions › New repository secret

Danh sách Secrets cần tạo hôm nay

- `DOCKERHUB_USERNAME`
- `DOCKERHUB_TOKEN`
- `EC2_HOST`
- `EC2_USERNAME`
- `EC2_SSH_KEY`

Vai trò của Container Registry trong CD

Tại sao không truyền source code lên EC2 rồi build trên đó?

Cách sai: build trực tiếp trên EC2

- EC2 quá tải **CPU / RAM** khi build
- Môi trường build **không đồng nhất**

Cách đúng: build ở Runner, đẩy qua Registry

Build trên Runner → **Docker Hub / ECR** →

EC2 chỉ pull về chạy

EC2 nhẹ nhàng, image đồng nhất ở mọi môi trường.

Cấu hình workflow: đăng nhập & push image lên Registry

push-image.yml — .github/workflows

```yaml
name: Build and Push
on:
  push:
    branches: [main]
jobs:
  build-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Log in to Docker Hub
        uses: docker/login-action@v3
      - name: Build and push
        uses: docker/build-push-action@v5
```

docker/login-action

Đăng nhập Registry bằng Secrets.

docker/build-push-action

Build image và push lên Registry trong một bước.

Bóc tách chi tiết file YAML Build & Push

push-image.yml · step đăng nhập

```yaml
- name: Log in to Docker Hub
  uses: docker/login-action@v3
  with:
    username : ${{ secrets.DOCKERHUB_USERNAME }}
    password : ${{ secrets.DOCKERHUB_TOKEN }}
```

Dùng **Docker Access Token** thay mật khẩu thật: an toàn hơn, có thể **revoke** ngay khi lộ.

Chiến lược đặt Tag cho Docker Image

`:latest`

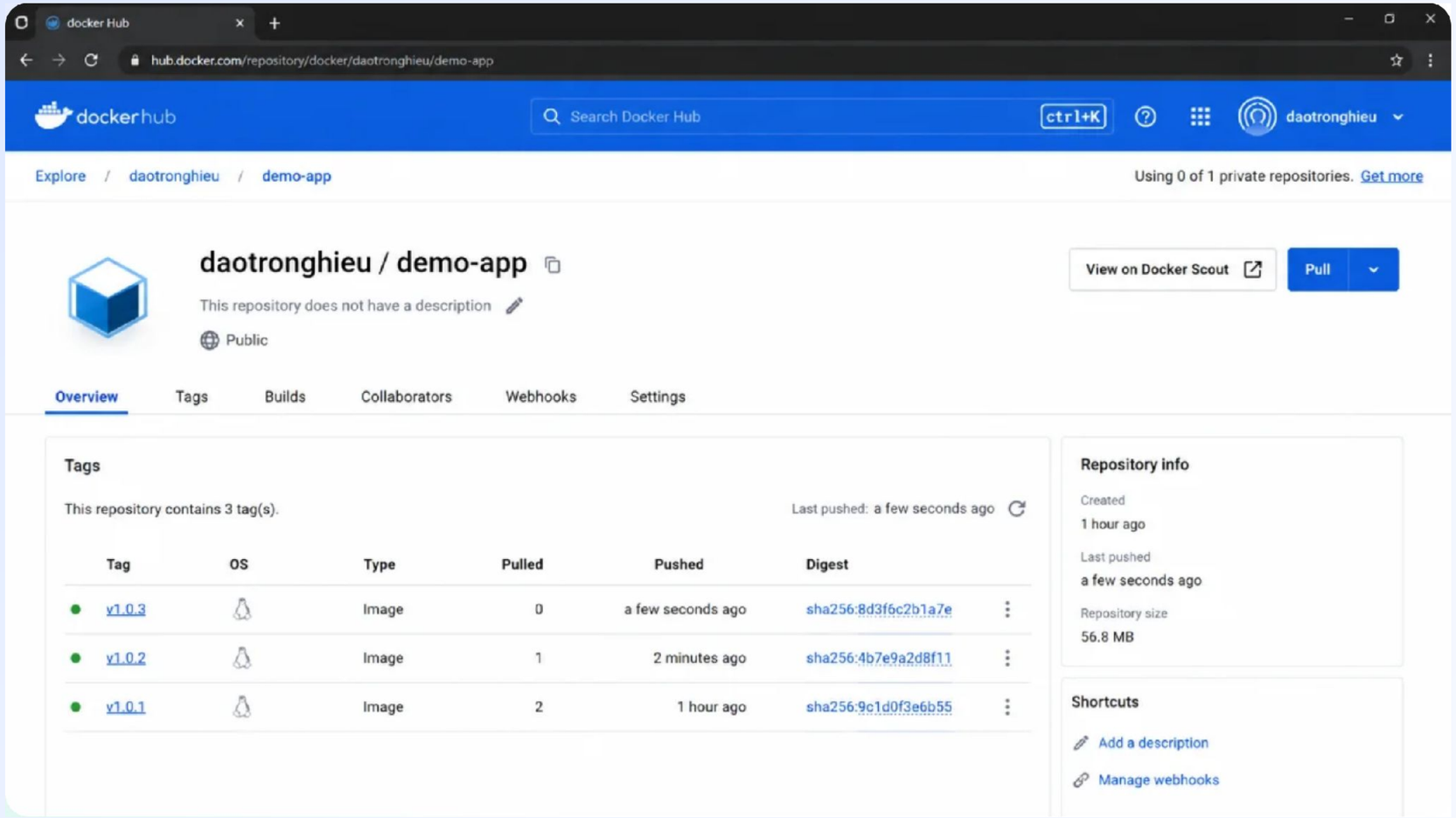
Luôn trỏ về bản build mới nhất. **Tiện lợi** nhưng **khó rollback** - không biết đang chạy bản nào.

`:<git-commit-sha>`

Mỗi lần build tạo **1 tag duy nhất** theo mã hash commit (github.sha) - truy vết và rollback chính xác.

☆ Best practice: gán **cả 2 tag** `latest` + `git-commit-sha` khi push.

Kiểm tra image đã lên Registry thành công



Docker Hub · trang repository của image

1 · Đọc log job Build and Push

Trên tab GitHub Actions, xác nhận step push báo xanh.

2 · Mở Docker Hub / ECR

Kiểm tra **Image Digest** và **Tags** vừa xuất hiện.

KHỐI 4 · Triển khai qua SSH lên AWS EC2 / VPS

Cơ chế xác thực SSH giữa GitHub Runner & AWS EC2

GitHub Runner giữ chìa

Private Key: copy toàn bộ nội dung file `.pem` dán vào Secret **`EC2_SSH_KEY`**.

SSH Port 22, không cần mật khẩu

EC2 giữ ổ khoá

Public Key: đã lưu sẵn trong `~/.ssh/authorized_keys` trên EC2 (tạo ở Buổi 8).

KHỐI 4 · Triển khai qua SSH lên AWS EC2 / VPS

Chuẩn bị môi trường trên AWS EC2 sẵn sàng cho CD

Checklist các thành phần bắt buộc, đã cài từ Buổi 8.

- **Docker & Docker Compose plugin** đã sẵn sàng.
- User chạy docker **không cần sudo**: `usermod -aG docker ubuntu`
- File `docker-compose.yml` định vị tại **thư mục dự án** trên EC2.

appleboy/ssh-action - cầu nối SSH

Action phổ biến nhất để **thực thi lệnh shell trên máy chủ từ xa** qua SSH - thay cho các thao tác gõ tay đã học ở Buổi 2.

`host`

IP / domain của EC2 - lấy từ `EC2_HOST`

`username`

User đăng nhập (vd. `ubuntu`)

`key`

Private Key từ `EC2_SSH_KEY`

`script`

Chuỗi lệnh shell sẽ chạy trên server

Phân tích file YAML Deploy SSH (deploy.yml)

deploy.yml .github/workflows

```yaml
- name: Deploy to EC2 via SSH
  uses: appleboy/ssh-action@v1.0.3
  with:
    host : ${{ secrets.EC2_HOST }}
    username : ${{ secrets.EC2_USERNAME }}
    key : ${{ secrets.EC2_SSH_KEY }}
    script: |
      cd /home/ubuntu/app
      docker compose pull
      docker compose up -d
      docker image prune -f
```

host / username / key

Thông tin kết nối, toàn bộ lấy từ GitHub Secrets.

script

`cd` vào thư mục dự án, `pull` image mới, `up -d` khởi động lại container.

Vấn đề tràn ổ cứng trên EC2 & lệnh dọn dẹp (Cleanup)

Vấn đề: Disk Full

Mỗi lần deploy, Docker tải image mới nhưng **image cũ vẫn tích tụ** - vài lần CD là đầy ổ đĩa.

Dung lượng đĩa 99%

Giải pháp: prune sau mỗi deploy

```
docker image prune -f
```

Xoá các **dangling images** không còn sử dụng - thêm vào cuối script deploy.

Dung lượng đĩa 30%

KHỐI 4 · Triển khai qua SSH lên AWS EC2 / VPS

Quản lý biến môi trường (.env) trên Staging / Production

⊗ Tuyệt đối **không push file .env lên Git**.

Cách 1

Tạo sẵn file .env trên EC2

SSH vào server, tạo file **thủ công 1 lần** tại thư mục dự án - đơn giản, phù hợp cấu hình ít thay đổi.

Cách 2

Ghi file .env động từ GitHub Secrets

Trong script SSH, dùng Secrets **ghi đè tạo file .env** ngay trước khi `docker compose up` - cấu hình luôn đồng bộ với repo.

KHỐI 5 · Testing, logs & xử lý sự cố

Kiểm thử kết quả sau Deploy (Post-Deployment Test)

Làm sao biết ứng dụng đã **thực sự live** thành công? Thêm step Healthcheck tự động:

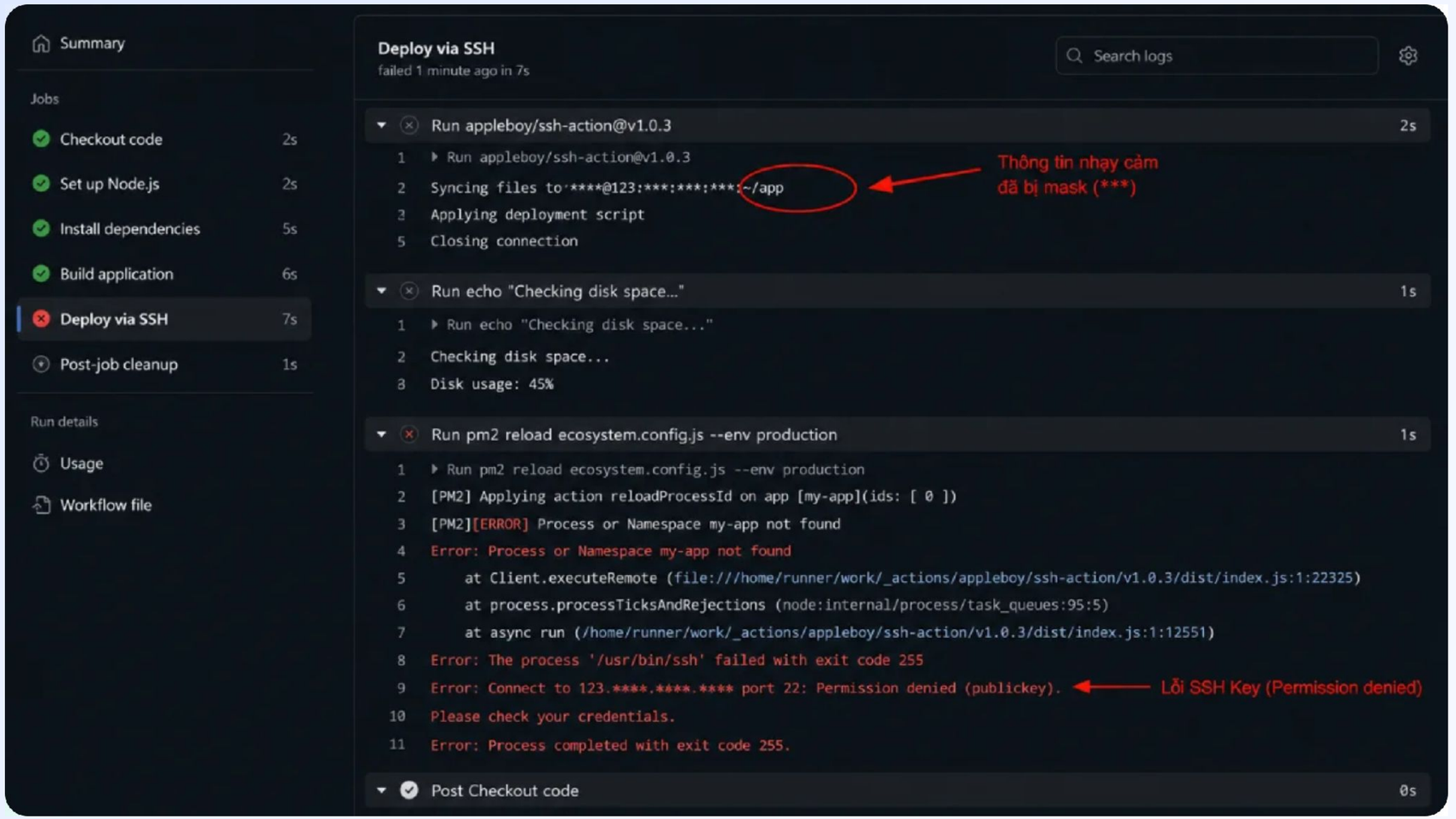
deploy.yml · step healthcheck

```yaml
- name: Healthcheck
  run : curl -f http://  ${{ secrets.EC2_HOST }}  :8080/health || exit 1
```

200 OK Ứng dụng live - workflow báo xanh.

Fail `exit 1` - workflow báo đỏ (Failed) ngay lập tức.

Kỹ năng đọc log trên GitHub Actions



GitHub Actions · log chi tiết từng step

Xem log từng step

Bấm vào từng step trong job để mở log chi tiết, tìm dòng báo lỗi đầu tiên.

Secret Masking *

GitHub biến mọi chuỗi khớp với Secrets thành *** - bằng chứng hệ thống che thông tin nhạy cảm.

KHỐI 5 · Troubleshooting #1

Lỗi Permission denied (publickey)

```
Error: Connect to xxx.xxx.xxx.xxx port 22: Permission denied (publickey)
```

Nguyên nhân 1

Private Key sai / thiếu

Copy thiếu dòng `-----BEGIN RSA PRIVATE KEY-----` khi dán vào Secret.

Nguyên nhân 2

IP EC2 bị đổi

Restart instance là đổi IP nếu không dùng **Elastic IP** - cập nhật lại `EC2_HOST`.

Nguyên nhân 3

Security Group chặn port 22

Chưa mở port 22 cho IP của GitHub Runner hoặc `0.0.0.0/0`.

KHỐI 5 · Troubleshooting #2

Lỗi Docker login failed & Container crash

Docker login failed

Sai username, hoặc dùng **mật khẩu thật** thay vì **Docker Access Token**.

Container crash (Exit Code 1 / 137)

Thiếu biến môi trường trong `.env`, hoặc ứng dụng **xung đột port** trên EC2.

Debug trực tiếp trên EC2

```
$ docker logs <container_id># hiển thị nguyên nhân lỗi của ứng dụng
```

Tổng kết Best Practices cho CI/CD Pipeline

Never hardcode secrets
Mọi biến bí mật đều nằm ở GitHub Secrets.

Immutable Artifacts
Build image 1 lần ở CI, dùng chính image đó để test và deploy.

Auto Cleanup
Luôn dọn dẹp Docker image rác sau khi deploy.

Fail Fast & Healthcheck
Báo lỗi ngay khi test hoặc healthcheck thất bại.